

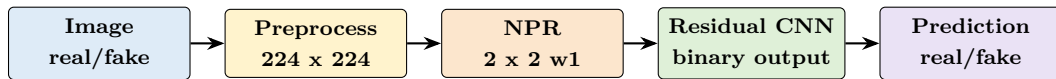
Egypt-Japan University of Science and Technology

Department of Computer Science and Engineering

CSE 429: Computer Vision and Pattern Recognition
Spring 2026 Final Project Report

Generalizable Deepfake Detection via Neighboring Pixel Relationships (NPR)

Replication and Evaluation of an Up-Sampling Artifact Detector



Name	ID
Farida Ehab	120220069
Mohamed Ehab	120220090
Nour Elhadidy	120220158
Sara Zeitoun	120220240
Nouran Yasser	120220250

Instructor: **Dr. Ahmed Gomaa**

Submission Date: **May 2026**

Contents

1	Abstract	1
2	Teaser Figure	1
3	Introduction	1
3.1	Motivation	1
3.2	Background on the Reference Paper	2
3.3	Project Goal	2
4	Approach	2
4.1	Overall Pipeline	2
4.2	NPR Representation	3
4.3	Implemented Model Architecture	3
4.4	Existing Implementations and What We Built	4
4.5	Design Choices and Justification	5
4.6	Obstacles and Fixes	5
5	Experiments and Results	5
5.1	Dataset and Experimental Setup	5
5.2	Final Test Sources	6
5.3	Evaluation Metrics	6
5.4	Phase 2 Validation Result	7
5.5	Final Phase 3 Overall Result on 8 Sources	7
5.6	Per-Source Results	8
5.7	Comparison with the Reference Paper	8
5.8	Trends and Interpretation	9
6	Qualitative Results	9
7	Conclusion and Future Work	11
8	References	12

1. Abstract

Synthetic images are becoming increasingly realistic, which creates a need for detectors that can identify fake images even when they come from unseen generators. This project addresses the problem of generalizable deepfake detection, where the detector is trained on one generator source but must work on other GAN-based image sources.

We replicated the Neighboring Pixel Relationships (NPR) idea from Tan et al. [1]. The method converts each image into a local pixel-difference representation that captures up-sampling artifacts, then trains a lightweight residual CNN to classify images as real or fake.

Our final fair test used the eight ForenSynths sources that overlap with the reference paper: ProGAN, StyleGAN, StyleGAN2, BigGAN, CycleGAN, StarGAN, GauGAN, and DeepFake. The final model achieved **92.22% pooled accuracy**, **97.31% average precision**, and **97.58% ROC-AUC** on **62,003 test images**. Its source-wise mean accuracy was **91.10%**, close to the paper’s reported **92.5%** mean accuracy.

2. Teaser Figure

Figure 1 shows the main project idea. Instead of sending the raw RGB image directly to the classifier, we first compute an NPR image. This representation emphasizes local neighbouring-pixel differences that may be caused by up-sampling operations inside image generators.

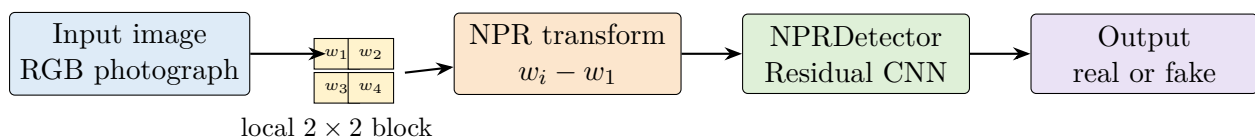


Figure 1: Teaser figure. The project uses local neighbouring-pixel differences as an artifact representation for generalizable real/fake image detection.

3. Introduction

3.1. Motivation

Modern generative models can create images that look natural to humans. This is useful in design, entertainment, and data generation, but it also makes it easier to spread fake media. For computer vision, the important question is not only whether a detector works on the generator it saw during training, but whether it can still detect fakes from unseen generators.

Many detectors learn features that are strongly connected to one generator or one dataset. This can lead to high validation accuracy but weak real-world generalization. For this

reason, the reference paper proposes studying common artifacts produced by the generator pipeline itself, especially up-sampling operations that appear in GAN and diffusion models [1].

3.2. Background on the Reference Paper

The reference paper is titled *Rethinking the Up-Sampling Operations in CNN-based Generative Network for Generalizable Deepfake Detection* [1]. Its main idea is that up-sampling layers create local pixel relationships in synthetic images. The paper proposes NPR as a simple representation that captures those local artifacts and uses them to train a binary real/fake detector.

In the paper, the detector is trained on the ForenSynths ProGAN four-class setting: car, cat, chair, and horse. It is then evaluated on unseen generator sources. For the eight ForenSynths benchmark sources, the paper reports a mean accuracy of about 92.5% and a mean average precision of 96.1% for NPR [1].

3.3. Project Goal

The goal of our project is to reproduce the NPR idea in a clear and deployable pipeline. The expected input to our system is a single RGB image. The desired output is a binary prediction: **real** or **fake**, together with the predicted probabilities.

4. Approach

4.1. Overall Pipeline

The project was implemented in phases so that each stage could be tested independently:

1. **Phase 1 - Preprocessing:** download and scan the ForenSynths/Wang_CVPR2020 dataset, then create clean CSV manifests.
2. **Phase 2 - Training:** implement the NPR transform and train a lightweight residual CNN for 10 epochs.
3. **Phase 3 - Evaluation:** evaluate the best checkpoint on the eight paper-matched test sources.
4. **Deployment preparation:** package the trained model, metadata, requirements, and inference script.

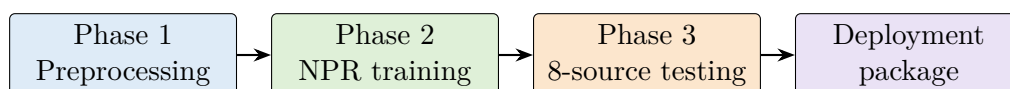


Figure 2: Project workflow used in the implementation.

4.2. NPR Representation

The NPR transform works on local image blocks. For a 2×2 block with pixels w_1, w_2, w_3, w_4 , the top-left pixel w_1 is used as the reference. The transformed block becomes:

$$\hat{v} = \{w_1 - w_1, w_2 - w_1, w_3 - w_1, w_4 - w_1\}.$$

This operation removes absolute pixel intensity and preserves local relative relationships. The reason it is useful is that generated images often contain local dependencies caused by up-sampling. NPR makes those dependencies easier for the classifier to learn.

Listing 1: Simplified NPR transform used in the project.

```
def npr_2x2_w1(x):
    # x shape: [batch, channels, height, width]
    ref = x[:, :, 0::2, 0::2]
    out = x.clone()
    out[:, :, 0::2, 0::2] = x[:, :, 0::2, 0::2] - ref
    out[:, :, 0::2, 1::2] = x[:, :, 0::2, 1::2] - ref
    out[:, :, 1::2, 0::2] = x[:, :, 1::2, 0::2] - ref
    out[:, :, 1::2, 1::2] = x[:, :, 1::2, 1::2] - ref
    return out
```

4.3. Implemented Model Architecture

Our model is a custom lightweight reproduction of the paper idea. It is not the exact official paper code, but it follows the same principle: NPR representation followed by a CNN-based binary classifier.

Table 1: Implemented NPRDetector architecture.

Stage	Operation	Output / purpose
Input preprocessing	RGB conversion, resize to 256, center crop to 224, ImageNet normalization	Tensor shape $3 \times 224 \times 224$
NPR transform	2×2 w1 neighbouring pixel relationship	Local artifact representation
Stem	3×3 convolution, 3 to 32 channels, stride 2, BatchNorm, ReLU	Initial feature extraction
Layer 1	2 residual BasicBlocks, 32 channels	Local NPR features
Layer 2	Residual stage, 32 to 64 channels, stride 2	Downsampling
Layer 3	Residual stage, 64 to 128 channels, stride 2	Higher-level artifacts
Layer 4	Residual stage, 128 to 256 channels, stride 2	Final feature map
Head	Adaptive average pooling + Linear(256, 2)	Real/fake logits

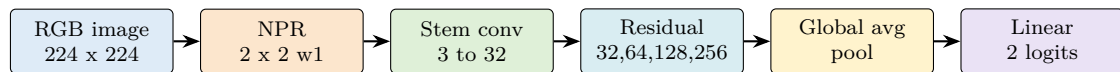


Figure 3: Architecture diagram of the implemented detector.

4.4. Existing Implementations and What We Built

The project used the paper as the scientific reference and reproduced its core NPR concept. The implementation itself was written in PyTorch. The preprocessing, manifest generation, model definition, training loop, evaluation code, per-source metric calculation, and deployment inference script were implemented by our team. We used standard libraries such as PyTorch, torchvision, NumPy, pandas, scikit-learn, and Pillow.

4.5. Design Choices and Justification

Table 2: Main parameters used in our implementation.

Parameter	Value and reason
Image size	Resize to 256 and center crop to 224, matching common CNN/vision preprocessing.
Normalization	ImageNet mean and standard deviation, used for stable CNN training.
NPR setting	2×2 w1 because the paper reports this as the main setting and it matches common 2x up-sampling.
Optimizer	Adam, learning rate 2×10^{-4} , following the reference paper’s training setting.
Batch size	32, matching the paper and fitting GPU memory.
Epochs	10 epochs in the final notebook.
Loss	Cross-entropy loss for binary classification.
Checkpoint selection	Best checkpoint selected by validation average precision.

4.6. Obstacles and Fixes

The first issue was that the initial test manifest was empty because the ForenSynths testing directory is organized by generator source rather than by the same car/cat/chair/horse class structure used in training. This was fixed by scanning each testing source folder and detecting its real/fake subfolders.

The second issue was fairness of comparison. The available testing folder contained more than the eight sources shown in the paper’s Table 1. To follow the paper benchmark exactly, the final report uses only the eight overlapping sources: ProGAN, StyleGAN, StyleGAN2, BigGAN, CycleGAN, StarGAN, GauGAN, and DeepFake.

5. Experiments and Results

5.1. Dataset and Experimental Setup

The project used ForenSynths/Wang_CVPR2020 from Kaggle. The training and validation sets follow the ProGAN four-class setting: car, cat, chair, and horse. The final test set uses the eight source folders that overlap with the reference paper’s ForenSynths benchmark.

Table 3: Dataset splits used in the project.

Split	Total	Real	Fake	Sources/classes	Notes
Train	144,024	72,012	72,012	car, cat, chair, horse	Balanced ProGAN training setting.
Validation	1,600	800	800	car, cat, chair, horse	Same-distribution validation.
Final test	62,003	31,006	30,997	8 generator sources	Cross-source evaluation matching paper Table 1.

5.2. Final Test Sources

Table 4: Eight final test sources used for fair comparison with the paper.

Source	Total images	Real	Fake
BigGAN	4,000	2,000	2,000
CycleGAN	2,642	1,321	1,321
DeepFake	5,405	2,707	2,698
GauGAN	10,000	5,000	5,000
ProGAN	8,000	4,000	4,000
StarGAN	3,998	1,999	1,999
StyleGAN	11,982	5,991	5,991
StyleGAN2	15,976	7,988	7,988
Total	62,003	31,006	30,997

5.3. Evaluation Metrics

We used the following evaluation metrics:

- **Accuracy:** percentage of correctly classified images.
- **Average Precision (AP):** ranking quality of fake probabilities across thresholds.
- **ROC-AUC:** ability to separate real and fake images over all thresholds.
- **Confusion matrix:** counts of true real, false fake, false real, and true fake predictions.
- **Precision, recall, and F1-score:** class-specific reliability for real and fake images.

A simple random baseline on this balanced binary task would be about 50% accuracy. Therefore, the final 92.22% pooled accuracy is significantly above the naive baseline.

5.4. Phase 2 Validation Result

Table 5: Phase 2 same-distribution validation result.

Metric	Value
Validation loss	0.001517
Validation accuracy	99.94%
Average precision	100.00%
ROC-AUC	100.00%
Confusion matrix	[[800, 0], [1, 799]]
Validation mistakes	1 / 1,600 images

This result confirms that the training pipeline, labels, model, and NPR transform are working correctly. However, this validation result is not the final generalization score because validation images are close to the training distribution.

5.5. Final Phase 3 Overall Result on 8 Sources

Table 6: Final pooled cross-source result on the eight paper-matched sources.

Metric	Value
Total evaluated images	62,003
Overall pooled accuracy	92.22%
Average precision	97.31%
ROC-AUC	97.58%
Real precision	92.34%
Real recall	92.10%
Real F1-score	92.22%
Fake precision	92.11%
Fake recall	92.35%
Fake F1-score	92.23%
Confusion matrix	[[28555, 2451], [2370, 28627]]

The result shows balanced behavior between real and fake classes. Real recall is 92.10%, and fake recall is 92.35%, meaning the final eight-source evaluation does not suffer from the strong fake-bias observed when additional non-paper sources were included.

5.6. Per-Source Results

Table 7: Per-source evaluation on the final eight test sources.

Source	Images	Accuracy	AP	ROC-AUC
BigGAN	4,000	85.68%	92.70%	94.06%
CycleGAN	2,642	90.61%	97.80%	98.11%
DeepFake	5,405	80.20%	94.65%	93.35%
GauGAN	10,000	77.37%	77.92%	83.78%
ProGAN	8,000	99.74%	100.00%	100.00%
StarGAN	3,998	100.00%	100.00%	100.00%
StyleGAN	11,982	96.85%	99.91%	99.88%
StyleGAN2	15,976	98.32%	100.00%	100.00%
Source-wise mean	–	91.10%	95.37%	96.15%

The strongest results appear on ProGAN, StarGAN, StyleGAN, and StyleGAN2. The weakest sources are GauGAN and DeepFake. This makes sense because some sources have image statistics and generation processes that differ more from the ProGAN training distribution.

5.7. Comparison with the Reference Paper

Table 8: Fair comparison with the paper’s Table 1 NPR results on the overlapping eight sources.

Source	Our accuracy	Paper NPR accuracy	Difference
BigGAN	85.68%	87.50%	-1.83
CycleGAN	90.61%	95.00%	-4.39
DeepFake	80.20%	77.40%	+2.80
GauGAN	77.37%	86.60%	-9.23
ProGAN	99.74%	99.80%	-0.06
StarGAN	100.00%	99.70%	+0.30
StyleGAN	96.85%	96.30%	+0.55
StyleGAN2	98.32%	97.30%	+1.02
Mean	91.10%	92.45%	-1.35

The paper reports the NPR method with a lightweight CNN of about 1.44M parameters and uses Adam with learning rate 2×10^{-4} and batch size 32 [1]. Our custom model has 1,614,882 trainable parameters. The final source-wise mean accuracy is close to the paper’s

value, with a gap of about 1.35 percentage points. This is a reasonable replication result because our implementation is a practical reproduction, not the exact official architecture and training environment.

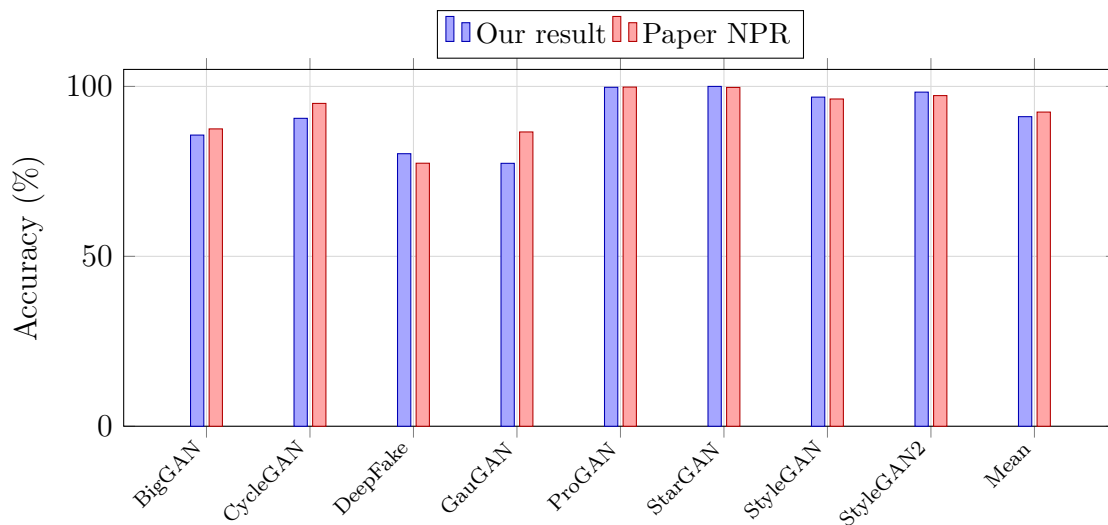


Figure 4: Accuracy comparison between our model and the paper’s NPR results.

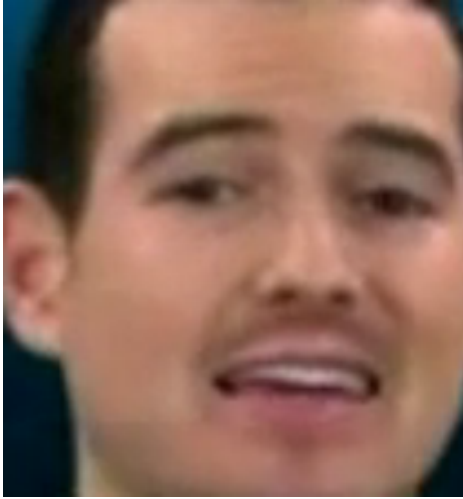
5.8. Trends and Interpretation

The final results reveal four important trends:

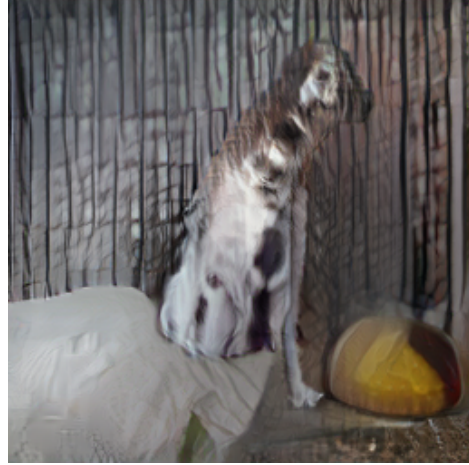
1. **Same-distribution validation is very high.** The 99.94% validation accuracy shows that the model learns the training task correctly.
2. **Cross-source testing is the true benchmark.** The final 8-source test accuracy of 92.22% is lower than validation but much more meaningful.
3. **Generalization is strong for several unseen GANs.** The detector performs very well on StarGAN, ProGAN, StyleGAN, and StyleGAN2.
4. **Some sources remain difficult.** GauGAN and DeepFake reduce the mean result, which suggests that domain shift and different image-generation pipelines still affect the detector.

6. Qualitative Results

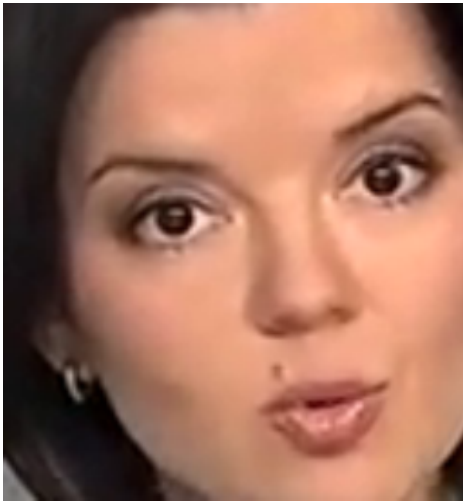
This section shows representative success and failure cases from the final eight-source evaluation. The CSV file `phase3_predictions.csv` stores the image path, true label, predicted label, fake probability, and correctness for every evaluated image. The selected examples include one correct real prediction, one correct fake prediction, one false positive, and one false negative.



(a) Correct real: DeepFake source, true real, predicted real.



(b) Correct fake: GauGAN source, true fake, predicted fake.



(c) False positive: DeepFake source, true real, predicted fake.



(d) False negative: GauGAN source, true fake, predicted real.

Figure 5: Qualitative examples from the final eight-source evaluation. The first row shows successful predictions, while the second row shows failure cases.

Table 9: Representative qualitative predictions from `phase3_predictions.csv`.

Case	Source	True label	Predicted label	Fake probability	Relative path
Correct real	DeepFake	real	real	0.000002	/testing/deepfake/0_real/919_0193.png
Correct fake	GauGAN	fake	fake	0.999169	/testing/gaugan/1_fake/000000060835.png
False positive	DeepFake	real	fake	0.998443	/testing/deepfake/0_real/381_0561.png
False negative	GauGAN	fake	real	0.358640	/testing/gaugan/1_fake/000000119516.png

The success cases show that the model can correctly detect both real and fake images from unseen testing folders. The failure cases are also important: a false positive means a real image was wrongly flagged as fake, while a false negative means a fake image was missed. These examples show why threshold calibration and source-wise error analysis are useful future improvements.

7. Conclusion and Future Work

This project successfully reproduced the main idea of NPR-based deepfake detection. The final model uses local neighbouring-pixel differences to expose up-sampling artifacts and classify images as real or fake. On the final eight-source test set, the model achieved 92.22% pooled accuracy, 97.31% average precision, and 97.58% ROC-AUC across 62,003 images.

The result is close to the reference paper’s Table 1 benchmark. Our source-wise mean accuracy was 91.10%, while the paper reports 92.45% over the same eight sources. This shows that the method was reproduced successfully in a practical project setting, even though our architecture is a custom implementation rather than the exact official model.

8. References

References

- [1] C. Tan, H. Liu, Y. Zhao, S. Wei, G. Gu, P. Liu, and Y. Wei, *Rethinking the Up-Sampling Operations in CNN-based Generative Network for Generalizable Deepfake Detection*, arXiv:2312.10461v2, 2023.
- [2] S.-Y. Wang, O. Wang, R. Zhang, A. Owens, and A. A. Efros, *CNN-generated images are surprisingly easy to spot... for now*, CVPR, 2020.
- [3] ForenSynths / Wang_CVPR2020 dataset, used for ProGAN training and cross-source fake image detection testing.
- [4] PyTorch documentation, used for model implementation, training, checkpoint saving, and inference.
- [5] Scikit-learn documentation, used for accuracy, average precision, ROC-AUC, confusion matrix, and classification report calculation.